

Intercity Carpooling App

Ride Start, OTP Pickup, Live Tracking, Drop-off, Dispute & Completion Design Document

Scope: This document covers the post-booking operational lifecycle. Ride publishing and booking are assumed to be already implemented.

Contents

- Purpose and scope
- Business scenario
- High-level architecture
- State machines
- End-to-end ride lifecycle
- OTP pickup verification
- Live tracking and family sharing
- Offline and server down handling
- Payment and payout logic
- Dispute service and review operations
- Edge cases
- API design
- Database model
- Frontend wireframes
- Admin dashboard
- MVP development checklist

1. Purpose and Scope

This document defines the development design for the ride operation lifecycle of an intercity carpooling app after a ride has already been published and booked.

- Each booking has its own pickup, onboard, drop-off, dispute, and payment lifecycle.
- The main ride can be completed only after all bookings are in a terminal state.
- Payments are handled per booking, so one disputed passenger does not block all driver payouts.
- OTP proves rider onboarding; geofence validates place; rider confirmation validates drop-off; delayed payout protects disputes.

2. Business Scenario

Example route used throughout the design:

Stop 1 -> Stop 2 -> Stop 3 -> Stop 4

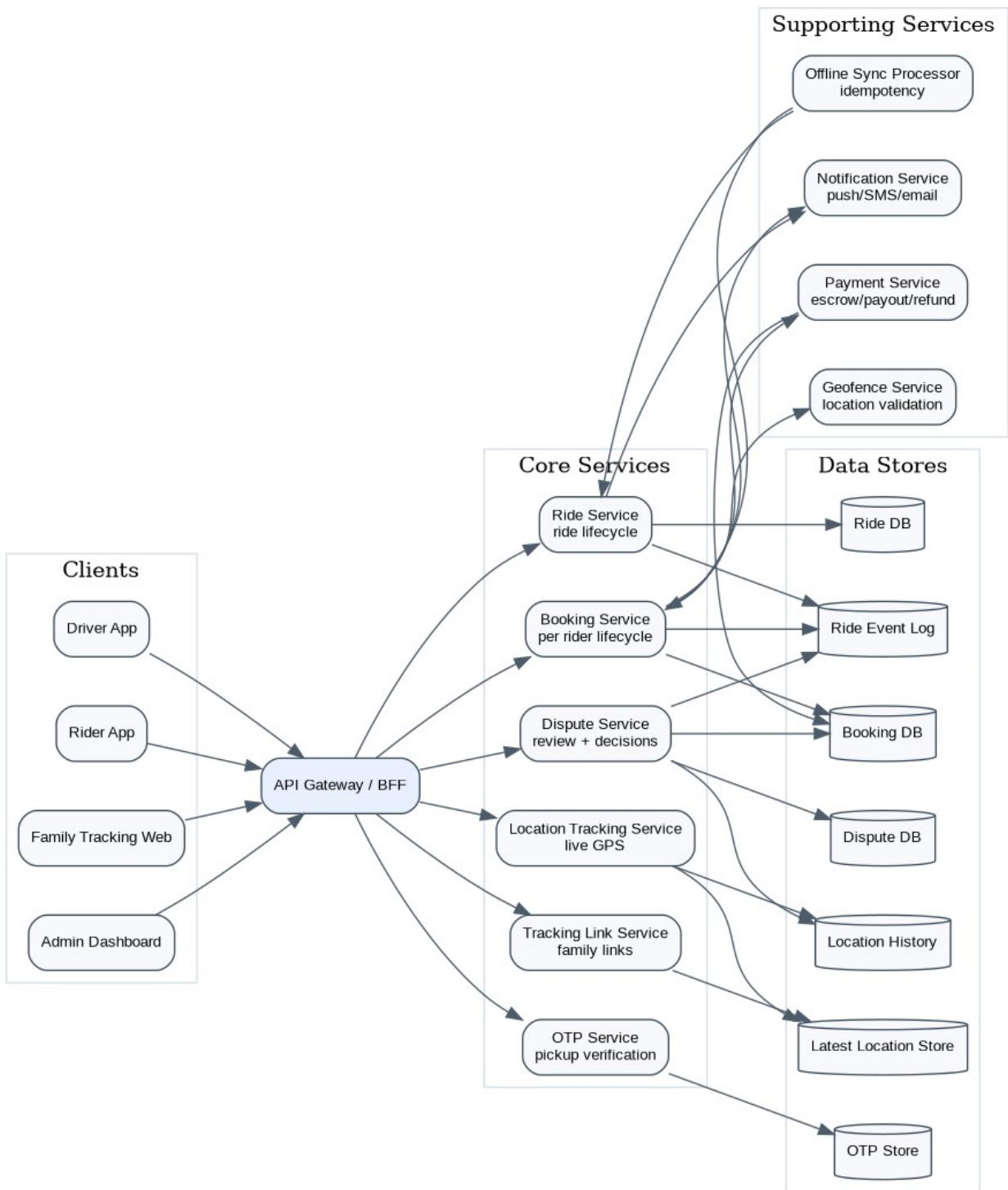
Rider	Pickup	Drop-off	Expected lifecycle
Rider 1	Stop 1	Stop 3	OTP pickup at Stop 1, drop at Stop 3
Rider 2	Stop 2	Stop 4	OTP pickup at Stop 2, drop at Stop 4
Rider 3	Stop 2	Stop 4	OTP pickup at Stop 2, drop at Stop 4

3. High-Level Architecture

The system separates ride lifecycle, rider booking lifecycle, live tracking, OTP, payments, offline sync, and disputes into clear services.

Figure 1 - High-level service architecture

Carpool Ride Start & Completion Design - rendered diagrams version

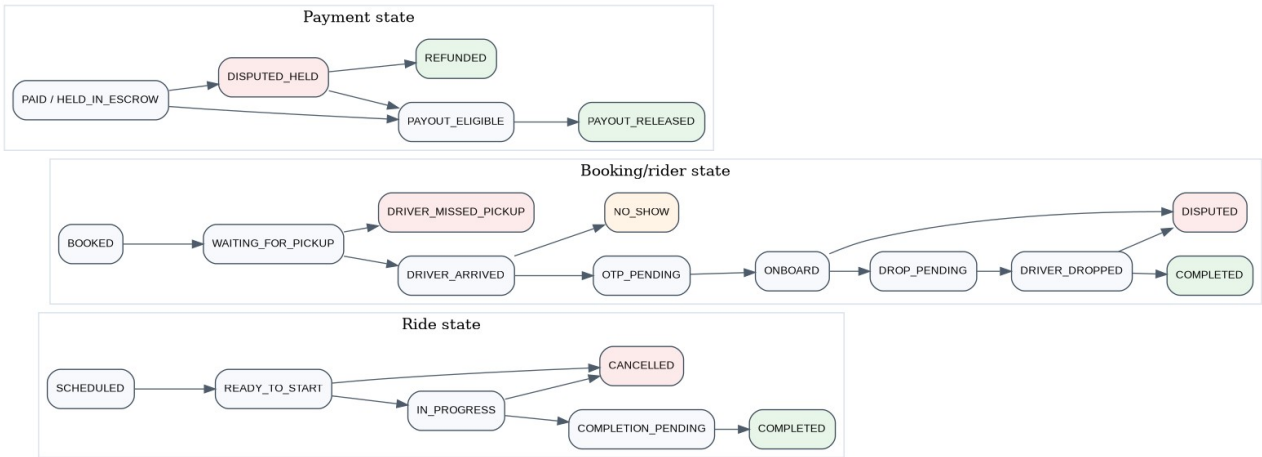


Service	Responsibility
Ride Service	Owns main ride status: start ride, finish ride, completion validation.
Booking Service	Owns per-rider booking states: arrived, OTP verification, onboard, drop-off, no-show.
OTP Service	Generates and verifies per-booking OTP for rider onboarding.
Location Tracking Service	Receives driver GPS updates and powers live tracking.
Geofence Service	Checks pickup/drop-off action against expected stop coordinates.
Payment Service	Keeps money in escrow and manages payout/refund eligibility.

Dispute Service	Collects evidence, applies rules/risk scoring, routes to manual review.
Offline Sync Processor	Processes queued mobile actions safely using idempotency keys.
Notification Service	Sends push/SMS/email for ride, pickup, drop-off, dispute, payment events.

4. State Machines

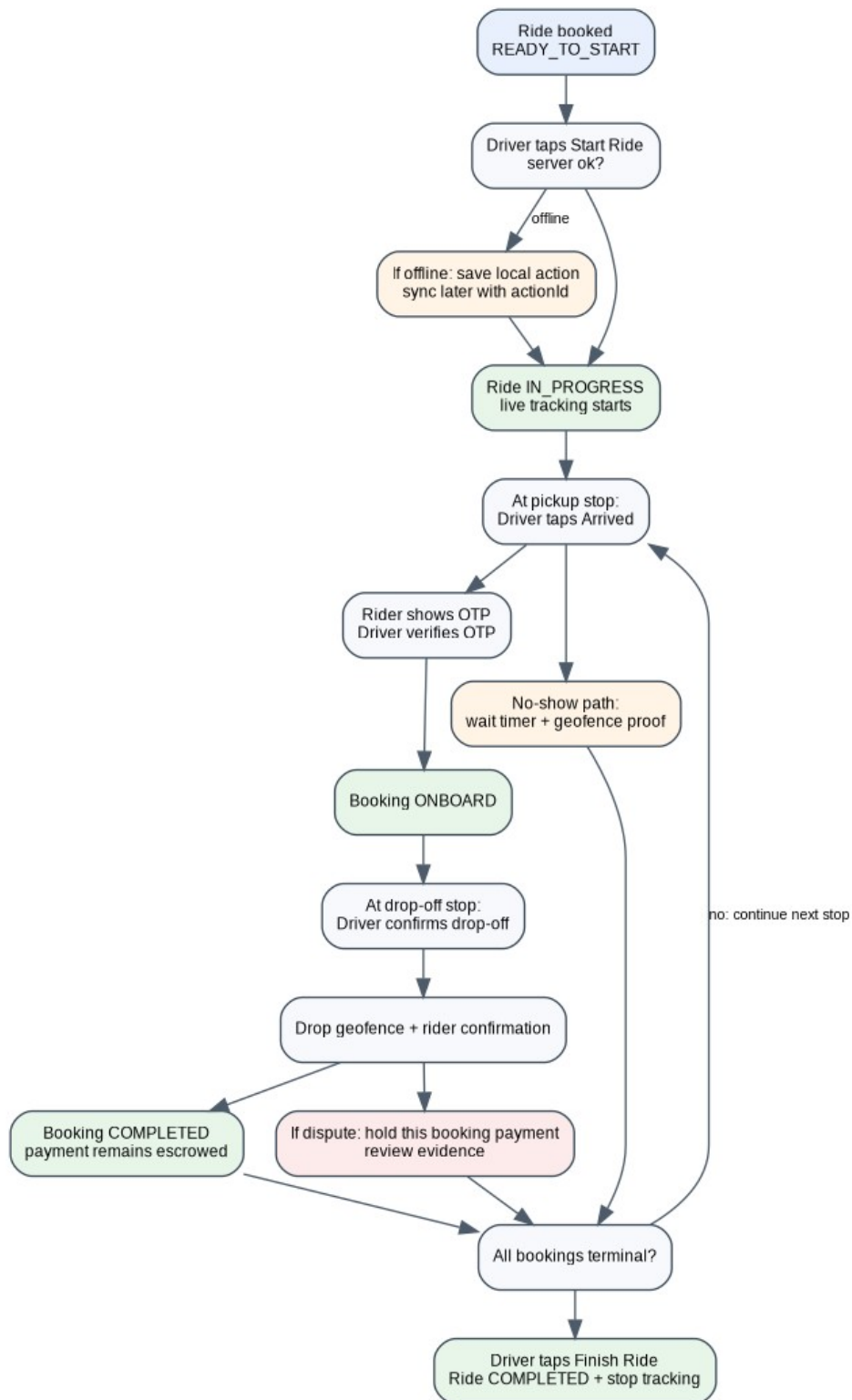
Figure 2 - Ride, booking, and payment states



Entity	Terminal states
Ride	COMPLETED, CANCELLED
Booking	COMPLETED, NO_SHOW, DRIVER_MISSED_PICKUP, CANCELLED, DISPUTED_RESOLVED_REFUND, DISPUTED_RESOLVED_PAYOUT
Payment	PAYOUT_RELEASED, REFUNDED

5. End-to-End Ride Lifecycle

Figure 3 - Complete ride start to completion flow



Step	Driver action	System result
Start	Tap Start Ride	Ride becomes IN_PROGRESS and location tracking starts.
Pickup	Tap Arrived, enter rider OTP	Booking becomes ONBOARD after OTP and geofence validation.
No-show	After wait timer, mark No-show	Booking becomes NO_SHOW; payment held during dispute window.
Drop-off	Tap Confirm Drop-off	Booking awaits rider confirmation and geofence validation.

Finish	Tap Finish Ride	Allowed only when all bookings are terminal.
--------	-----------------	--

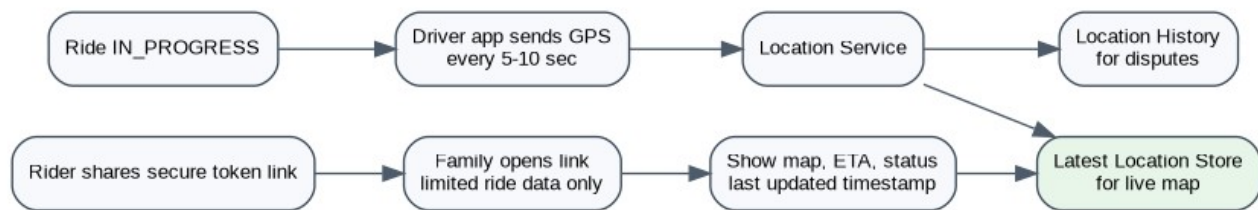
6. OTP Pickup Verification

OTP is generated per booking and shown only to the rider. The driver must enter the OTP to onboard the rider. This prevents the driver from accidentally marking the wrong person as picked up.

Validation	Rule
OTP correct	Hash matches active booking OTP and OTP is not used.
Ride state	Ride must be IN_PROGRESS.
Booking state	Booking must be WAITING_FOR_PICKUP, DRIVER_ARRIVED, or OTP_PENDING.
Driver authorization	Driver must be assigned to the ride.
Geofence	Driver should be within 100-300m of pickup point, otherwise mark suspicious and require confirmation.

7. Live Tracking and Family Sharing

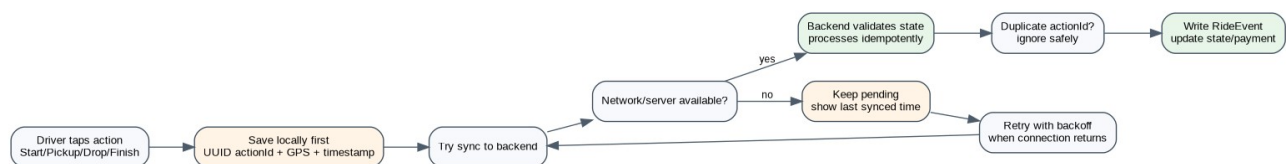
Figure 4 - Live tracking and family sharing flow



The rider can share a secure token link with family. The family tracking page should not expose other riders' personal information. It should show only approximate location, ETA, ride status, and last updated timestamp.

8. Offline and Server Down Handling

Figure 5 - Offline-safe action sync



All critical driver actions must be saved locally before the app attempts server sync. The backend must process every action idempotently using actionId.

Offline action payload field	Purpose
actionId	UUID for idempotency and duplicate prevention.
rideId / bookingId	Target ride and optionally target rider booking.
actionType	START_RIDE, DRIVER_ARRIVED, VERIFY_OTP, MARK_NO_SHOW, CONFIRM_DROPOFF, FINISH_RIDE.
clientTimestamp	When the action happened on the device.
GPS lat/lng/accuracy	Evidence for geofence and dispute review.
syncStatus	PENDING, SYNCED, FAILED.

9. Payment and Payout Logic

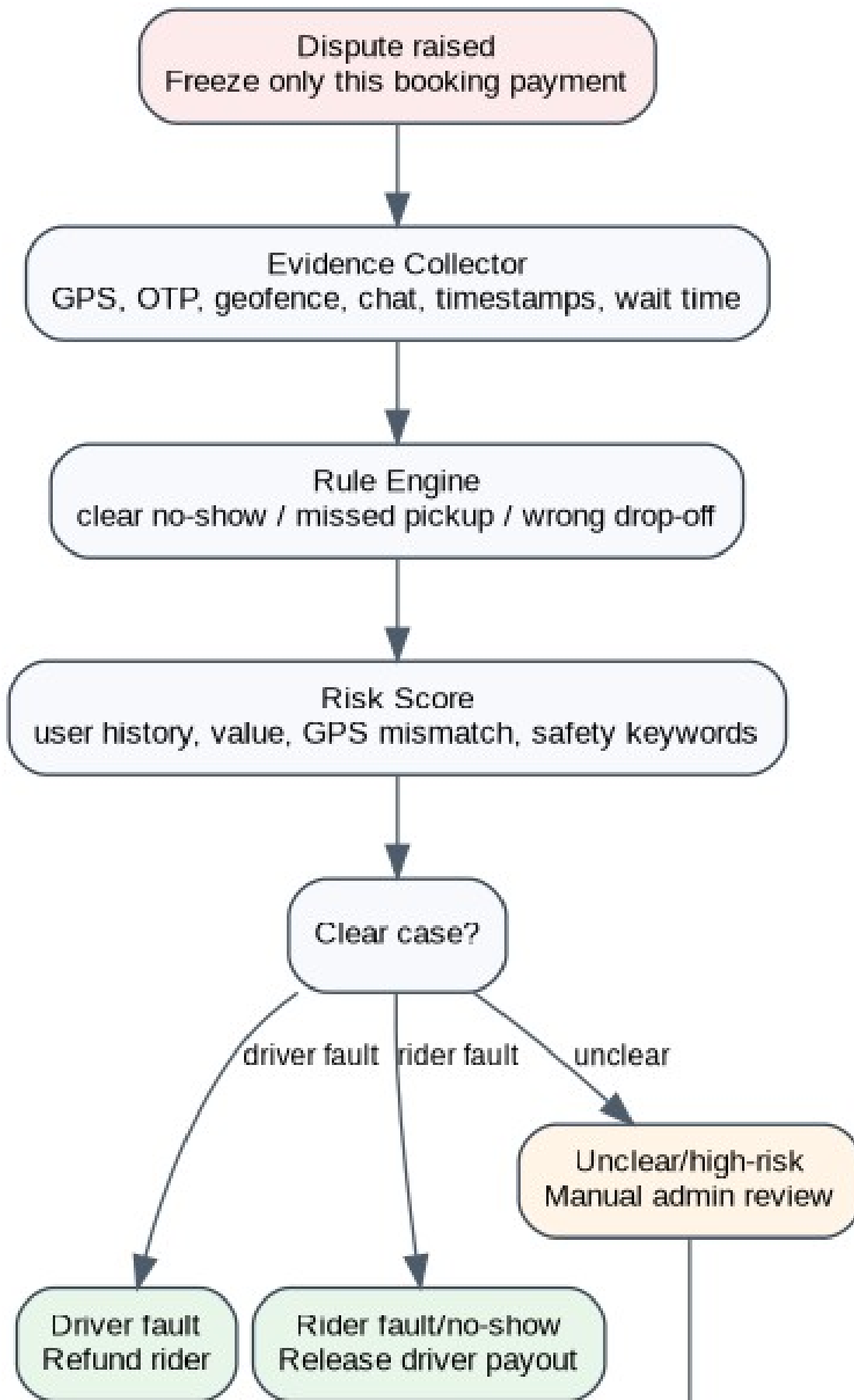
- Rider payment is captured during booking and held in escrow.

- Driver action alone never releases money immediately.
- A completed booking becomes payout eligible only after validation and dispute window.
- A disputed booking payment is held independently; other completed bookings can continue toward payout.

Situation	Rider outcome	Driver outcome
Normal completed trip	No refund	Payout eligible after dispute window
Rider no-show	No/partial refund based on policy	Payout eligible if wait/geofence proof is valid
Driver missed pickup	Full refund	No payout for that booking
Unclear dispute	Payment held	Payment held until review

10. Dispute Service and Review Operations

Figure 6 - Dispute review and auto-resolution flow



The dispute system must avoid manual review for every case. Clear cases should be auto-resolved using evidence and rules; unclear, high-risk, or safety-related cases go to admin review.

Evidence	Example use
Driver GPS	Was driver near pickup/drop-off when action was performed?
Rider GPS	Was rider actually at pickup point when claiming missed pickup?
OTP status	Was onboarding verified?
Geofence result	Was action inside allowed radius?
Wait timer	Did driver wait required time before no-show?
Chat/call attempts	Did either side attempt contact?
User history	Repeated no-shows, false disputes, driver complaints.
Case	Auto-decision rule
Rider no-show	Driver was near pickup, waited required time, OTP not verified, rider not near pickup, no valid dispute.
Driver missed pickup	Rider was near pickup, driver never reached pickup, OTP not verified, report raised quickly.
Wrong drop-off	Usually manual review if geofence failed and rider disputes.
OTP verified but rider disputes pickup	Usually driver-favorable unless safety/fraud keywords are present.

11. Edge Cases

Edge case	Expected handling
Rider does not show	Driver taps Arrived, waits required time, marks no-show, payment held during dispute window.
Driver skips rider	Rider reports missed pickup; system checks rider GPS, driver GPS, OTP absence; refund if driver fault.
Wrong OTP	Allow limited retries; after limit, require support/manual verification.
Rider phone dead	Allow manual verification flow; keep payment held until post-confirmed or reviewed.
Driver GPS unavailable	Allow action with warning; require rider confirmation; mark event suspicious.
Server down	Save action locally and sync later with idempotency key.
App killed during ride	On reopen, fetch ride state, resume location tracking, sync pending actions.
Driver finishes too early	Backend rejects finish if any booking is ONBOARD, WAITING, or unresolved.
Multiple riders same pickup	Each rider has separate OTP; driver verifies each booking separately.
Multiple riders same drop-off	Confirm drop-off per booking; payment eligibility handled per booking.

12. API Design

Endpoint	Purpose
POST /rides/{rideId}/start	Start ride and begin operational lifecycle.
POST /rides/{rideId}/finish	Finish ride after all bookings are terminal.
POST /bookings/{bookingId}/driver-arrived	Mark driver arrived at pickup.
POST /bookings/{bookingId}/verify-pickup-otp	Verify rider OTP and mark ONBOARD.
POST /bookings/{bookingId}/mark-no-show	Mark rider no-show after wait timer.
POST /bookings/{bookingId}/confirm-dropoff	Driver confirms rider drop-off.
POST /bookings/{bookingId}/rider-confirm-dropoff	Rider confirms successful drop-off.
POST /bookings/{bookingId}/report-missed-pickup	Rider reports driver missed pickup.
POST /rides/{rideId}/locations	Driver app sends location update.
POST /bookings/{bookingId}/tracking-link	Create secure family tracking link.
POST /offline-actions/sync	Sync queued mobile actions.

13. Database Model

Entity	Key fields
--------	------------

Ride	id, driverId, routeId, status, actualStartTime, actualEndTime, currentStopSequence
Stopover	id, rideId, sequence, name, lat, lng, plannedArrivalTime, geofenceRadiusMeters
Booking	id, rideId, riderId, pickupStopId, dropoffStopId, status, paymentStatus, otpStatus
RideEvent	id, rideId, bookingId, actionId, eventType, actorType, lat, lng, timestamps, validationStatus
LocationUpdate	rideId, driverId, lat, lng, speed, heading, accuracy, timestamp
TrackingLink	id, bookingId, tokenHash, expiresAt, accessScope
OTP	id, bookingId, otpHash, expiresAt, usedAt, attemptCount, status
Dispute	id, rideId, bookingId, raisedBy, reason, status, evidence, resolution

14. Frontend Wireframes

14.1 Driver Screens

TODAY'S RIDE

Hamburg -> Tallinn

Status: READY TO START

Passengers: 3 booked

Stop 1: Pickup Rider 1

Stop 2: Pickup Rider 2, Rider 3

Stop 3: Drop Rider 1

Stop 4: Drop Rider 2, Rider 3

[START RIDE]

PICKUP RIDER

Rider: Rider 1

Stop 1 -> Stop 3

Geofence: Valid

Distance: 50m

Enter OTP: [_ _ _ _]

[VERIFY OTP]

[MARK NO-SHOW AFTER TIMER]

DROP-OFF

Rider: Rider 1

Destination: Stop 3

Geofence: Valid

[CONFIRM DROP-OFF]

RIDE SUMMARY

Completed: 3

No-show: 0

Disputes: 0

Expected earnings: 42 EUR

[FINISH RIDE]

+-----+

14.2 Rider Screens

+-----+

| *UPCOMING RIDE* |

+-----+

Pickup: Stop 2 Time: 09:00

Driver: Michael Vehicle: BMW 320

[LIVE TRACKING] [SHARE TRACKING]

+-----+

+-----+

| *DRIVER APPROACHING* |

+-----+

ETA: 5 min

OTP: 8437

[I AM AT PICKUP] [DRIVER DID NOT PICK ME UP]

+-----+

+-----+

| *ARRIVED* |

+-----+

Driver marked drop-off. Did you arrive safely?

[YES, CONFIRM] [REPORT ISSUE]

+-----+

15. Admin Dashboard

+-----+

| *DISPUTE REVIEW* |

+-----+

Reason: Driver did not pick me up

Recommendation: Refund Rider Confidence: 87%

Evidence:

- Rider GPS valid: 45m from pickup

- Driver GPS missing: 1.8km away

- OTP not verified

- Rider reported within 30 minutes

[REFUND RIDER] [PAY DRIVER] [SPLIT] [ESCALATE]

+-----+

16. MVP Development Checklist

- Backend state machine for Ride, Booking, Payment, Dispute.

Carpool Ride Start & Completion Design - rendered diagrams version

- OTP generation and verification per booking.
- Driver timeline screen with one primary action at a time.
- Live tracking updates every 5-10 seconds after ride starts.
- Secure family tracking link.
- Geofence validation for pickup and drop-off.
- Offline action queue with idempotent backend processing.
- Per-booking payment escrow and payout eligibility.
- Dispute evidence collection, auto-resolution rules, and admin queue.
- Admin review dashboard for unclear disputes.